

# *Code Optimization*

# *Optimization*



- In computing, optimization is the process of modifying a system to make some aspect of it work more efficiently or use fewer resources.
- Ex: a computer program may be optimized so that it executes more rapidly or is capable of operating with less memory storage or other resources, or draw less power.
- The system may be a single computer program, a collection of computers or even an entire network such as the internet.



# *Code Optimization*

- **Optimization** is a program transformation technique, which tries to improve the code that consume less resources (i.e. CPU, Memory) and deliver high speed.
- In optimization, high-level general programming constructs are replaced by very efficient low-level programming codes.
- Usually done at the end of the development stage since it reduces readability & adds code that is used to improve the performance.

# *Code Optimization*



## **Optimized code's features:-**

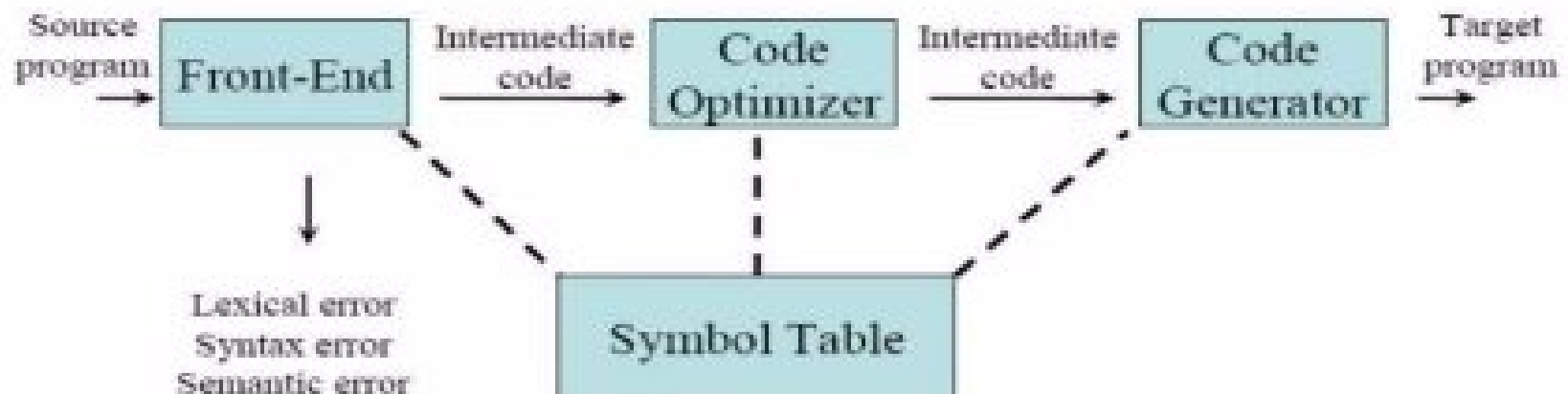
- Executes faster
- Code size get reduced
- Efficient memory usage
- Yielding better performance
- Reduces the time and space complexity



# Code Optimization

Optimizations are classified into two categories:

- **Machine independent optimizations** - improve target code without taking properties of target machine into consideration.
- **Machine dependent optimization** - improve target code by checking properties of target machine.



# *Code Optimization*



## ***Criteria for Optimization:-***

- An optimization must preserve the meaning of a program:
  - Cannot change the output produced for any input.
  - Can not introduce an error.
- Optimization should, on average, speed up programs.
- Optimization should itself be fast and should not delay the overall compiling process.

# *Code Optimization*



Optimization can occur at several levels:

- 1. Design level:-** At the highest level, the design may be optimized to make best use of the available resources.
  - The implementation of this design will benefit from the use of suitable efficient algorithms and the implementation of these algorithms will benefit from writing good quality code.
- 2. Compile level:-** Use of an optimizing compiler tends to ensure that the executable program is optimized at least as much as the compiler can predict

# *Code Optimization*

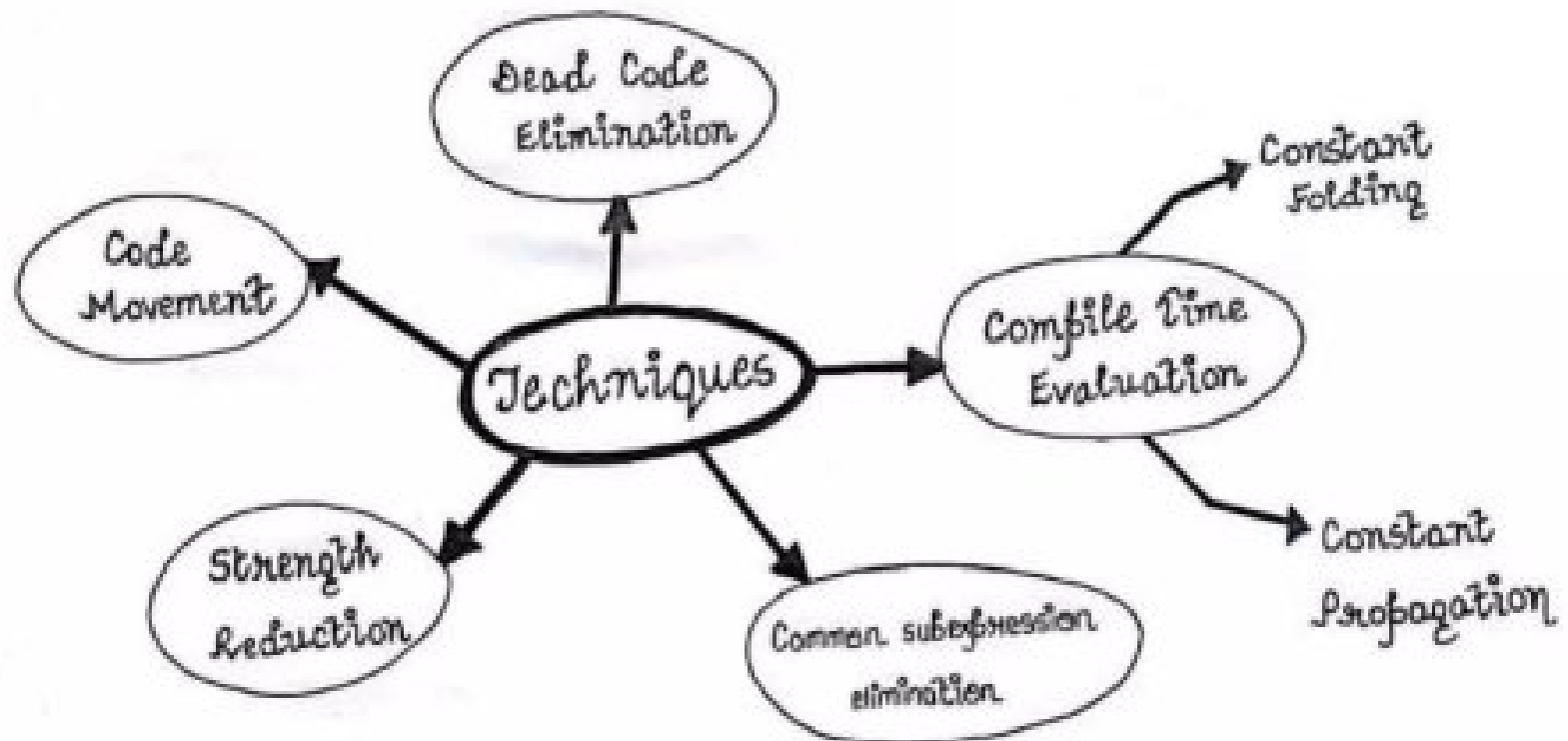


***Improvements can be made at various phases:-***

- **Source Code:** Algorithm's transformation can produce spectacular improvements
- **Intermediate Code:** Compiler can improve loops, procedure calls and address calculations.
  - Typically only optimizing compilers include this phase
- **Target Code:** Compilers can use registers efficiently

# Code Optimization

- Main techniques of Optimization:-



# *Code Optimization*



***There are primarily 3 types of optimizations:-***

- (1) Local optimization - Apply to a basic block in isolation
- (2) Global optimization - Apply across basic blocks
- (3) Loop optimization – Apply on loops

Another significant optimization can be:-

- (4) peep-hole optimization - Apply across boundaries

# *Code Optimization*

- Optimization performed within a basic block.
- This is simplest form of optimizations
- No need to analyze the whole procedure body.
- Just analyze the basic blocks of the procedure.
- The local optimization techniques include:
  - Constant Folding
  - Constant Propagation
  - Algebraic Simplification
  - Operator Strength Reduction
  - Dead Code Elimination

# *Local Code Optimization*



## **A) Constant folding-**

- As the name suggests, this technique involves folding the constants by evaluating the expressions that involves the operands having constant values at the compile time.
- Ex: Circumference of circle =  $(22/7) \times \text{Diameter}$
- Replace  $(22/7)$  with 3.14 during compile time and save the execution time.

# *Local Code Optimization*



## ***B) Constant Propagation-***

- In this technique, if some variable has been assigned some constant value, then it replaces that variable with its constant value in the further program wherever it has been used during compilation, provided that its value does not get alter in between.
- Ex:  $\pi = 3.14$ ,  $\text{radius} = 10$
- $\text{Area of circle} = \pi \times \text{radius} \times \text{radius}$
- Substitutes constants value during compile time and saves the execution time.

# Local Code Optimization

## C) Algebraic Simplification-

- Use algebraic properties to simplify expressions
- Some expressions can be simplified by replacing them with an equivalent expression that is more efficient.

• Ex:

$-(-i)$



$i$

```
void f (int i)
{
    a[0] = i + 0;
    a[1] = i * 0;
    a[2] = i - i;
    a[3] = 1 + i + 1;
}
```



```
void f (int i)
{
    a[0] = i;
    a[1] = 0;
    a[2] = 0;
    a[3] = 2 + i;
}
```

# Code Optimization



## D) Strength Reduction-

- As the name suggests, this technique involves reducing the strength of the expressions by replacing the expensive and costly operators with the simple and cheaper ones

| Code before Optimization | Code after Optimization |
|--------------------------|-------------------------|
| $B = A \times 2$         | $B = A + A$             |

Here, the expression “ $A \times 2$ ” has been replaced with the expression “ $A + A$ ” because the cost of multiplication operator is higher than the cost of addition operator.

# Code Optimization



## ***E) Dead code elimination-***

- Those code are eliminated which either never executes or are not reachable or even if they get execute, their output is never utilized.

| Code before Optimization                                       | Code after Optimization |
|----------------------------------------------------------------|-------------------------|
| <pre>i = 0 ;<br/>if (i == 1)<br/>{<br/>a = x + 5 ;<br/>}</pre> | <pre>i = 0 ;</pre>      |

# Code Optimization



## ***F) Common sub-expressions elimination:***

- Eliminates the redundant expressions and avoids their computation again and again.

| Code before Optimization                                                                              | Code after Optimization                                            |
|-------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------|
| <pre>S1 = 4 x i S2 = a[S1] S3 = 4 x j S4 = 4 x i // Redundant Expression S5 = n S6 = b[S4] + S5</pre> | <pre>S1 = 4 x i S2 = a[S1] S3 = 4 x j S5 = n S6 = b[S1] + S5</pre> |

# *Global Code Optimization*



- Optimization across basic blocks within a procedure /function
- Could be restricted to a smaller scope, Example: a loop
- Data-flow analysis is done to perform optimization across basic blocks
- Each basic block is a node in the flow graph of the program.
- These optimizations can be extended to an entire control - flow graph
- Most of compiler implement global optimizations with well founded theory and practical gains

# *Peephole Optimization*



- Optimization technique that operates on the one or few instructions at a time.
- Performs machine dependent improvements
- Peeps into a single or sequence of two to three instructions (peephole) and replaces it by most efficient alternative (shorter or faster) instructions.
- Peephole is a small moving window on the target systems

# *Peephole Optimization*

Characteristics of peep-hole optimizations:-

- Redundant-instruction (loads and stores)elimination
- Flow-of-control optimizations - - Elimination of multiple jumps (i.e. goto statements)
- Elimination of unreachable code
- Algebraic simplifications
- Reducing operator strength
- Use of machine idioms

replace **Add #1,R**  
by **Inc R**

# *Loop Optimization*



- Loop optimization plays an important role in improving the performance of the source code by reducing overheads associated with executing loops.
- Loop Optimization can be done by removing:
  - Loop invariant
  - Induction variables

# Loop Optimization



- Removal of Loop Invariant data:-

```
i = 1
```

```
s = 0
```

```
do{
```

```
s = s + i
```

```
a = 5
```

```
i = i + 1
```

```
{
```

```
while (i <= n)
```



```
i = 1
```

```
s = 0
```

```
a = 5
```

```
do{
```

```
s = s + i
```

```
i = i + 1
```

```
{
```

```
while (i <= n)
```

Bringing `a=5` outside the do while loop, is called code motion.

# Loop Optimization

- Induction variables:-

```
i = 1
s = 0
S1 = 0
S2 = 0
while (i <= n)
{
s = s + a[ i ]
t1 = i * 4
s = s + b[ t1 ]
t2 = t1 + 2
s2 = s2 + c[ t2 ]
i = i + 1
}
```



```
i = 1
s = 0
S1 = 0
S2 = 0
t2 = 0
while (i <= n)
{
s = s + a[ i ]
t1 = t1 + 4
s = s + b[ t1 ]
s2 = s2 + c[ t1 + 2 ]
i = i + 1
}
```



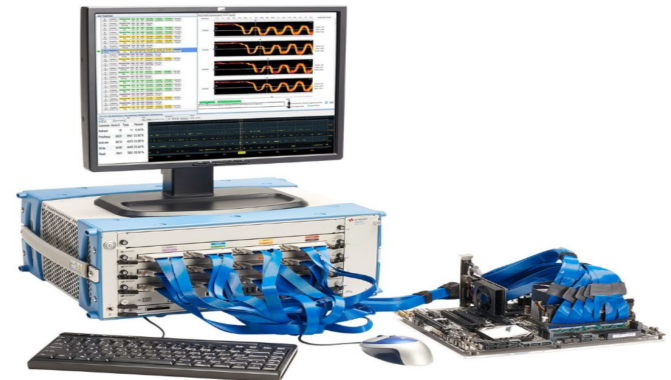
“+” replaced “\*”,  
t1 was made  
independent of i

t1, t2 are induction variables. i is inducing t1 and t1 is inducing t2

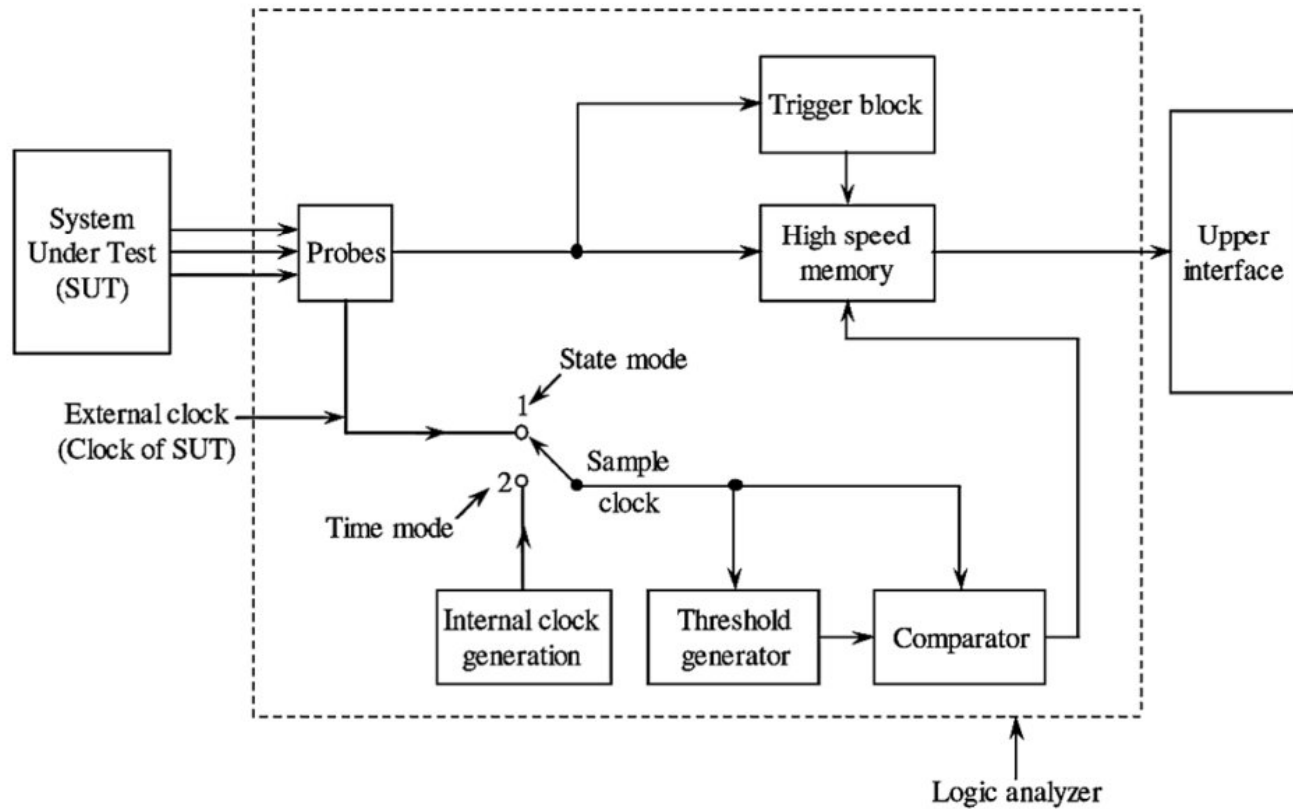
# Logic Analyzer and Programming Environment

# LOGIC ANA

- A instrument that deals with the digital systems for their troubleshooting purposes is known as **logic analyser**.
- Logic analyser is important to digital data domain, such data domain is characterized by state space concepts, data formats, data flows and equipment architecture.
- **A logic analyzer is a device that captures the data of the digital system and provide the display to the captured data for analysis.**



# BLOCK DIAGRAM OF LOGIC ANALYZER

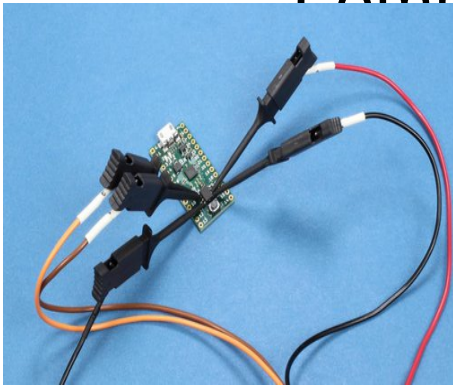


# LOGIC ANALYZER - MAIN COMPONENTS

- Probes
- Internal clock generator – to sample signals at predefined frequency, independent of SUT
- Threshold generator

Comparator

C  
e



# LOGIC ANALYZER - MAIN COMPONENTS

- Appropriate **probes** are used to connect the data lines of a digital system (i.e., System Under Test (SUT)) to the logic analyzer.
- The **internal clock generator** from a built-in oscillator generates the clock pulses which are used for sampling the input data.
- The **comparator** block compares the sampled data and a specified threshold voltage generated by threshold generator.
- The **memory** of the logic analyzer is used to store the data captured by logic analyzer.

# WORKING OF LOGIC ANALYZER

- After receiving the signal from the system under test SUT, it samples the input signal at certain clock frequency.
- In the **sampling process**, it compares the input signal with a certain threshold voltage at each sample point (i.e., at each clock pulse) and depending upon the result of comparison it converts the input signal at that sampling instant into a logic 1 (High) or logic 0 (low) state.
- If the level of input signal **is higher than the threshold voltage** it is recognized as **logic-1** and if its **level is less than the threshold level** it is recognized as **logic 0**.
- Thus, by sampling process the input signal gets converted into a pattern of 0's and 1's (i.e., the sampled data is a series of 0's and 1's).
- The sampled data is then stored in the **memory of logic analyzer** with each sample point occupying a single memory location.

# TYPES OF LOGIC ANALYZER

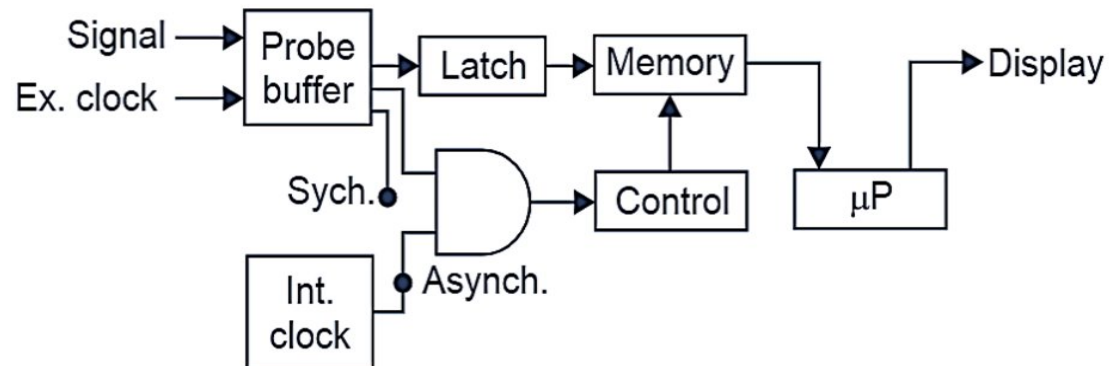
- The logic Analyzer may be of two types:
  1. Logic Timing Analyzer (Asynchronous Measurements)
  2. Logic Analyzer (Synchronous Measurement)

# LOGIC TIMING ANALYZER

- This Analyzer “samples” the waveform at each edge of the internal clock generated at regular intervals of time, stores them in memory and displays the same in a timing waveform.
- This is called as “asynchronous” measurement since the internal clock runs asynchronous to the logic system being tested.

# LOGIC ANALYZER SYNCHRONOUS MEASUREMENT

- This uses a signal from the circuit under test, external clock determines when the logic signal is to be stored. This is called as “synchronous” measurement since the external clock runs synchronous to the logic system being tested.



# MODERN LOGIC ANALYZER

- Modern logic analyzer combines both the above functions in one unit.
- It combines both functions i.e., asynchronous and synchronous. The logic signal data to be measured enters the Analyzer through “Probe Buffer”.
- The output from the buffer is clocked into a latch at regular intervals. The latched data is stored in the memory and is displayed by the microprocessor ( $\mu$ P).
- The signal data used to latch can be obtained from two different sources.
- For asynchronous operation, an internal clock is used and for synchronous operation, an external clock is used.

# APPLICATION OF LOGIC ANALYZER

- A logic analyzer has the ability to capture and display the data of multiple digital signals (usually more than eight) at a time.
- Hence, a logic analyzer can be used to examine a large number of digital signal lines of a digital system.

# APPLICATION OF LOGIC ANALYZER

- Signal analysis troubleshooting
- Communication protocol debugging
- Timing anomaly detection
- Execution flow visualization
- Software-hardware interaction analysis
- Comprehensive testing scenario
- Data integrity evaluation
- Validation of Complex algorithm

# PROGRAMMING ENVIRONMENT

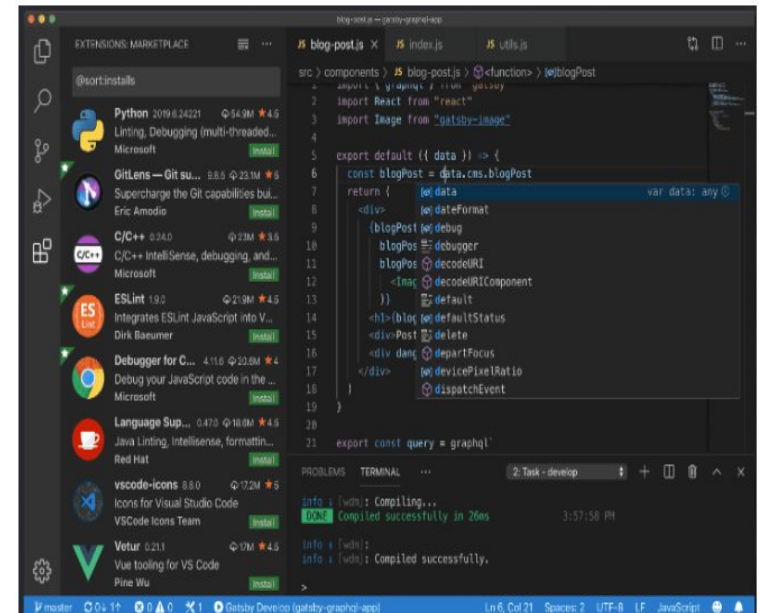
- In a general sense, a programming environment combines hardware and software that allows a developer to build applications.
- Developers typically work in integrated development environments or IDEs.
- These connect users with all the features necessary to write and test their code correctly. Different IDEs will offer other capabilities and advantages.

# INTREGATED DEVELOPMENT ENVIRONMENT

- IDEs – also known as integrated development environments – afford developers a convenient workspace when embarking on a project by packaging all the development tools a developer needs into a single graphical user interface.
- They typically provide a space for source code editing and a debugging program that locates problems in any code that has been written.
- Additionally, IDEs offer automation functionality that can handle complaining, packaging, and testing code.
- The popularity of IDEs in the development community hinges on their convenience. While integrating components can offer a fuller app customization experience, using an IDE provides a streamlined development process while minimizing the potential for user error.

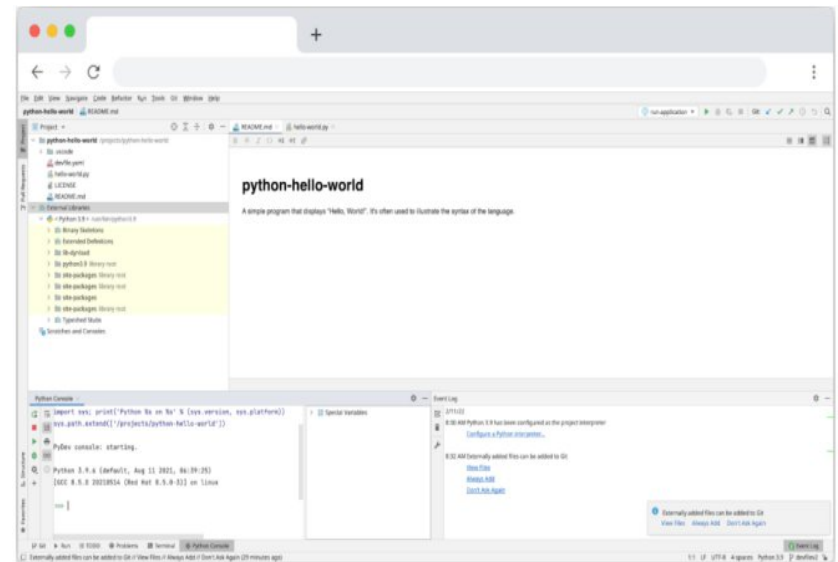
# VISUAL STUDIO

- Visual Studio is an IDE developed by Microsoft. It offers a great deal of versatility, providing users with an expansive library of extensions that allows for more customization than other environments. **Compatibility testing** in Visual Studio is also difficult to match.
- While the environment is well-suited for in-depth projects, it takes a lot of memory to open and run. Between this strain on your system and the high price point of this IDE, firms tackling small-scale projects should look to more lightweight, open-source options.



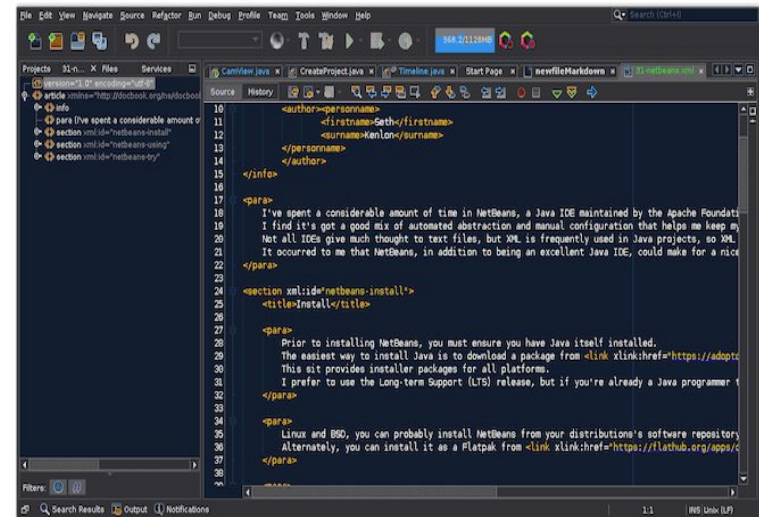
# Eclipse

- Another open-source IDE, Eclipse can offer a wide array of functionality to users. It sets itself apart from other environments via the **extensive plugin catalog** it offers. Its growing online knowledge repository is also a plus.
- Although the number of features Eclipse offers may seem to only be a positive aspect of the environment, it can be intimidating to new or inexperienced developers.



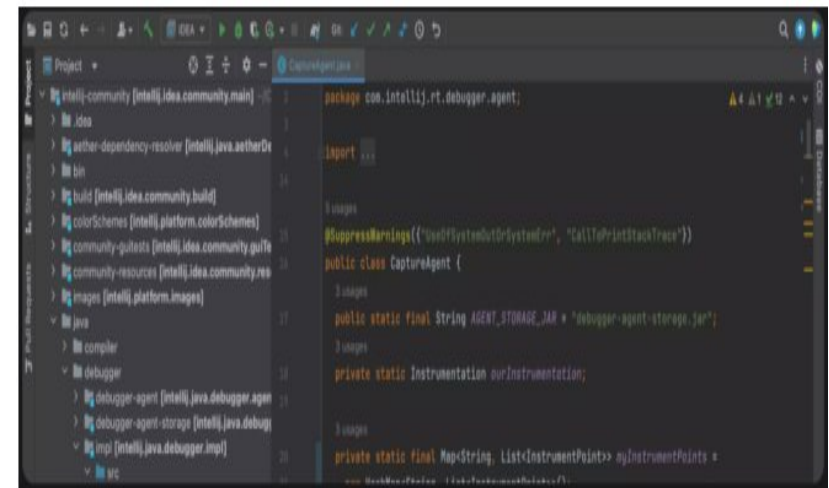
# NetBeans

- Primarily used for Java projects, NetBeans is a great open-source option for both neophytes and seasoned developers. Its **drag-and-drop user interface** as well as its number of **project templates** make it ideal for all levels of experience.
- The biggest difficulty users encounter with NetBeans is its potential for slow performance on some systems. The environment also requires that extensions be downloaded, to be compatible with nonJava projects.



# IntelliJ

- With a wide range of **price options** and a number of potential features, IntelliJ is an intriguing IDE choice for any development project. The environment is compatible with many platforms and offers a strong **tool suite to mobile app** developers.
- Despite its adaptability, IntelliJ is not the best choice for beginners. Between its complicated UI and plethora of shortcuts, it can pose a challenge to anyone not familiar with it or other IDEs.



# Programming Environment Selection Matters

- Just as the proper equipment is necessary when manufacturing a tangible product, building an app calls upon developers to choose the best possible programming environment. By combining a number of development tools, integrated development environments are strong options when developing a solution.
- However, your understanding of your **project specifications** in conjunction with your **chosen IDE features** will dictate the overall success of your efforts.